

Case Study 1: Real-time Digital Payments Platform

Scenario: A digital payments company like paytm/phx processes a million transactions per minute. Transactions must be validated, checked for fraud, updated in accounts and stored for auditing.

Kafka Architecture Design:

Producers (payment services) publish topics like 'transaction-raw' which are divided into many partitions across multiple brokers. Different consumer groups read independently for parallel real-time processing.

Partition Design for High Throughput:

- Use a high number of partitions and key by user-id or transaction-id to ensure parallelism and per-user ordering.
- Each partition has a leader and replicas and data is committed only after all in-sync replicas acknowledge.
- The controller monitors health and manages partition leader elections when failure occurs.
- Each message has a unique offset, and consumers commit offsets after successful processing.
- Use replication factor=3 to tolerate one broker failure while maintaining durability and availability.

Case Study 2: E-commerce Order Processing System (Using Kafka Architecture)

Scenario: An e-commerce platform like Amazon process orders through multiple services. Each service must process the same order event independently.

- A single 'orders' topic is created, and each service belongs to a separate consumer group so all the services receive and process the same order independently.
- Partitioning splits the 'orders' topic into multiple partitions, allowing parallel consumption and faster order processing across multiple instances.
- If the inventory consumer fails, only its consumer group stops processing while other services continue unaffected.
- Kafka ensures durability by writing messages to leader and replicating them to in-sync replicas before acknowledging the producer.
- Order service (producer) → Kafka Topic
(with partitioning & replication)



Multiple Consumer Groups

(payment, inventory, shipping, Notifications)

processing the same event independently

Case Study 3: Smart City IoT Monitoring

Scenario: A smart city deploys 50,000 IoT sensors monitoring traffic, pollution and temperature. Data must be processed in real time for alerts and stored for analytics.

- Kafka handles continuous high-volume sensor data by distributing message across multiple partitions and brokers, enabling parallel ingestion and real-time stream processing.
- Partitions should be distributed evenly across all brokers so ^{each} broker handles a balanced share of traffic, pollution and temperature data load.
- Zookeeper stores and manages cluster metadata such as broker registration, topic configuration and partition leadership information.
- If a broker handling traffic goes offline, one of its in-sync replicas on another broker becomes the new leader and processing continues without data loss.
- Zookeeper detects broker failure through heartbeats (ephemeral znodes) and immediately notifies the Kafka controller.
- When a leader broker fails, the controller (with Zookeeper coordination) elects a new leader from the in-sync replicas to maintain availability.

Q2

Case Study 4: Distributed Banking Core System

Scenario: A banking system runs across multiple servers. Only one server must act as master for transaction coordination to avoid inconsistencies.

- In a zookeeper ensemble, servers elect a leader through a majority voting process where the server with the latest transaction ID and majority support becomes the leader. Other acts as follower.
- Ephemeral znodes are temporary nodes that automatically gets deleted when a server session ends, helping detect crash instantly.
- If the leader fails, followers detect the missing heartbeat and initiate a new election to choose a leader with majority quorum.
- Zookeeper implements distributed locking by allowing servers to create sequential ephemeral znodes and granting the lock to the one with the smallest sequence number.
- Quorum ensures that more than half of the ensemble nodes agree on updates, maintaining consistency and preventing split-brain scenarios.

(Case Study 5: OTT Streaming Analytics Platform)

Scenario: An OTT platform track user activity. Data is streamed for Real-time recommendation, Trend analytics, Advertisement targeting

- User activity (play, pause, search like) act as producers and publish events asynchronously to kafka topics via kafka producer APIs
- Each analytics service belongs to a separate consumer group, allowing them to read and process the same stream independently.
- Replications stores multiple copies of each partition across brokers, so if one broker fails, another replica takes over and process the same stream independently.
- Zookeeper coordinates kafka brokers by maintaining metadata about brokers, topics, partitions and leader assignment
- During broker startup, the broker registers itself in zookeeper, syncs cluster metadata and participates in leader election if required.

Q9